

Unit Testbenches

Self-checking verification for the compact divider and the I-cache

Two self-checking Icarus Verilog testbenches verify the blocks the team added. Both compare the design under test against a golden reference model on directed and randomised stimulus, and print **ALL TESTS PASSED** on success (and **\$finish** with a non-zero error count on failure), so they drop straight into a regression flow. The reference model is written independently of the RTL, so the test does not simply restate the design.

Tooling note: these use **iverilog** and **vvp** from the bundled **oss-cad-suite**. If they are not already on the PATH:

```
export PATH="$PWD/../oss-cad-suite/bin:$PATH" # from picorv32/, adjust as needed
```

Divider testbench: div_tb.v

This drives **picorv32_pcp_div** over the PCPI handshake, issues each operation and waits for the result, then checks it against an exact RV32M golden model. It covers **div**, **divu**, **rem** and **remu**, including divide-by-zero and the **INT_MIN / -1** overflow case, with directed vectors (both sign combinations and the edge cases) plus 4000 random operations, 4054 checks in total.

The reference takes the signed operations through signed temporaries (**sa**, **sb**) so that Verilog does not silently compute them unsigned, which was the cause of an early false failure where the reference, not the hardware, was wrong:

```
case (f3)
  F_DIV : golden = (b==0) ? 32'hFFFFFFFF // x / 0 = -1
          : (a==32'h80000000 && b==32'hFFFFFFFF) ? 32'h80000000 // overflow
          : sa / sb; // sa, sb signed
  F_REM : golden = (b==0) ? a
          : (a==32'h80000000 && b==32'hFFFFFFFF) ? 32'd0
          : sa % sb;
  F_DIVU: golden = (b==0) ? 32'hFFFFFFFF : a / b;
  F_REMU: golden = (b==0) ? a : a % b;
endcase
```

Run it from the **picorv32/** directory:

```
make div_test
```

I-cache testbench: picosoc/icache_tb.v

For the cache, both the data returned and the hit-or-miss timing are correctness properties, and the look-ahead interface has to be driven with exactly the right timing. The testbench exercises the two shipping caches, **icache_multiword_lookahead** and **icache_multiword_lookahead_2way**, at the port interface:

- **Behavioural memory model** with a runtime-adjustable fill latency. Every suite runs at latency 1 and 3 to stress the fill state machine.
- **Replicates the PicoRV32 look-ahead protocol**. The look-ahead address is presented one cycle before the real request, which is what lets the cache flop the synchronous block-RAM read into **la_cached_*** in time for a single-cycle hit.
- **Golden reference model** mirroring the RTL tag, index and valid state, and for the two-way cache the **lru_array** and **replace_way_next** logic. Each fetch is checked two ways: returned data must equal **mem[addr>>2]**, and hit or miss must match the model, so any divergence in which way is filled surfaces as a later hit-or-miss mismatch.

- **Directed cases:** cold-fill and re-hit, sequential walk, conflict (direct-mapped thrash against two-way coexistence), two-way LRU eviction, and pass-through with no pollution, on top of 2000 random fetches per pass.

The per-fetch check itself is short:

```
exp = mem[addr>>2]; // golden data
pred_hit = ref_predict_hit(addr); // model of tag/valid/LRU, before this access
got = do_fetch(addr); // drive the one-cycle-early look-ahead, then fetch
if (got != exp) fail("data");
if (meas_hit != pred_hit) fail("hit/miss");
ref_update(addr, pred_hit); // advance the model exactly as the RTL would
```

The testbench is parameterised at `LINES=8`, `WORDS_PER_LINE=4` (small, to force evictions quickly). Bump the parameters to exercise the production 128x16 and 64x16 geometries. Run it from `picosoc/`:

```
cd picosoc
make icache_test # builds tb_dm + tb_2way, runs both
```

Results

All checks pass for both blocks (Icarus Verilog 14.0, oss-cad-suite):

Testbench	Coverage	Result
<code>div_tb</code> (divider)	4054 ops (directed + 4000 random)	PASS
<code>icache_tb</code> direct-mapped	directed + 2000 random × 2 latencies	PASS
<code>icache_tb</code> two-way LRU	directed + 2000 random × 2 latencies	PASS

Capacity versus associativity, a note on a fair comparison

`make icache_test` also runs a 2000-fetch random stream and reports the hit rate. It is tempting to read the direct-mapped against two-way numbers as "associativity helps", but `LINES` means different things for the two modules. For the two-way cache it is the number of sets per way, so the same `LINES` gives the two-way cache twice the total capacity. The testbench therefore runs three configs: direct-mapped, two-way at equal `LINES` (twice the capacity), and two-way at equal capacity (half the `LINES`).

Config (same random stream, latency 1)	Capacity	Hits / 2000	Hit rate
Direct-mapped, <code>LINES=8</code>	32 words	246	12.3%
Two-way, <code>LINES=8</code> (equal <code>LINES</code>)	64 words	479	23.9%
Two-way, <code>LINES=4</code> (equal capacity)	32 words	237	11.8%

Two takeaways. The headline near 2 times hit rate at equal `LINES` is a capacity effect, twice the storage, not associativity. At equal capacity, on a uniformly random address stream the two are essentially tied (12.3% against 11.8%), because associativity only removes conflict misses and a structureless random stream has none. So this unit testbench validates functional correctness and capacity scaling, and it is not the right vehicle to measure the associativity benefit. That benefit shows up on a real benchmark with the cache deliberately undersized to about half the instruction hot loop, where the two-way cache cuts conflict misses and measures about 14% lower CPI than direct-mapped at the same capacity (see the Benchmark Design document).